



Il Sistema Operativo iOS

Sistemi Operativi

Ivano Bilenchi

Ingegneria Informatica e dell'Automazione
(A.A. 2023/2024)

Indice

- Introduzione, concetti e filosofia
- App, architettura e gestione degli eventi
- Architettura dello stack userland
- Architettura del kernel
- Driver e filesystem
- Boot chain
- Feature di sicurezza

Cos'è iOS?



- Sistema operativo mobile sviluppato da **Apple Inc.**
- Impiegato su **iPhone** e **iPod Touch**.
- Nelle varianti **iPadOS**, **watchOS** e **tvOS**, è anche utilizzato su **iPad**, **Apple Watch** e **Apple TV**. *→ VARIANTI DI iOS*
- Basato su **macOS**, sistema operativo desktop **UNIX-like**.
- **Closed source**, *→ GUI, DRIVER ...* ma con diverse componenti **open**:
 - **Darwin**: il sistema operativo su cui si basano macOS, iOS, watchOS e tvOS, assieme al suo kernel **XNU**, è open source (fatta eccezione per i device driver). Dal 2017 sono open anche le componenti ARM.
 - **Framework**: diversi framework inclusi in iOS sono open source (WebKit, Foundation, Bonjour, FoundationDB, ecc.)
 - **Tool e linguaggi**: **Swift** ed il runtime di **Objective-C** sono open, oltre che gran parte dei tool di sviluppo (LLVM, LLDB, Clang, ecc.)

S.O., KERNEL, COMPONENTE... TUTTI OPEN SOURCE !

Concetti e filosofia



- **Sistema app-centric:** tutto è un'app, anche l'interfaccia utente principale (**SpringBoard**).
- **Metafore di interazione:** interazione diretta con **tocco e gesti** (tap, swipe, pan, pinch, ecc.) invece che mediata da dispositivi di puntamento.
- **Design:** linee guida curate da Apple (**HIG**, Human Interface Guidelines), interfacce intuitive e consistenti, focus sull'accessibilità.
→ *OBBLIGAZIONE PER ENTRARE NELL'APP STORE*
- **Privacy e sicurezza:** distribuzione centralizzata del software, aggiornamenti di sistema first-party, supporto crittografico e biometrico, policy di **raccolta dati** minimale.
→ *INSTALLAZIONE DIFFICILE DA TERZE PARTI!*

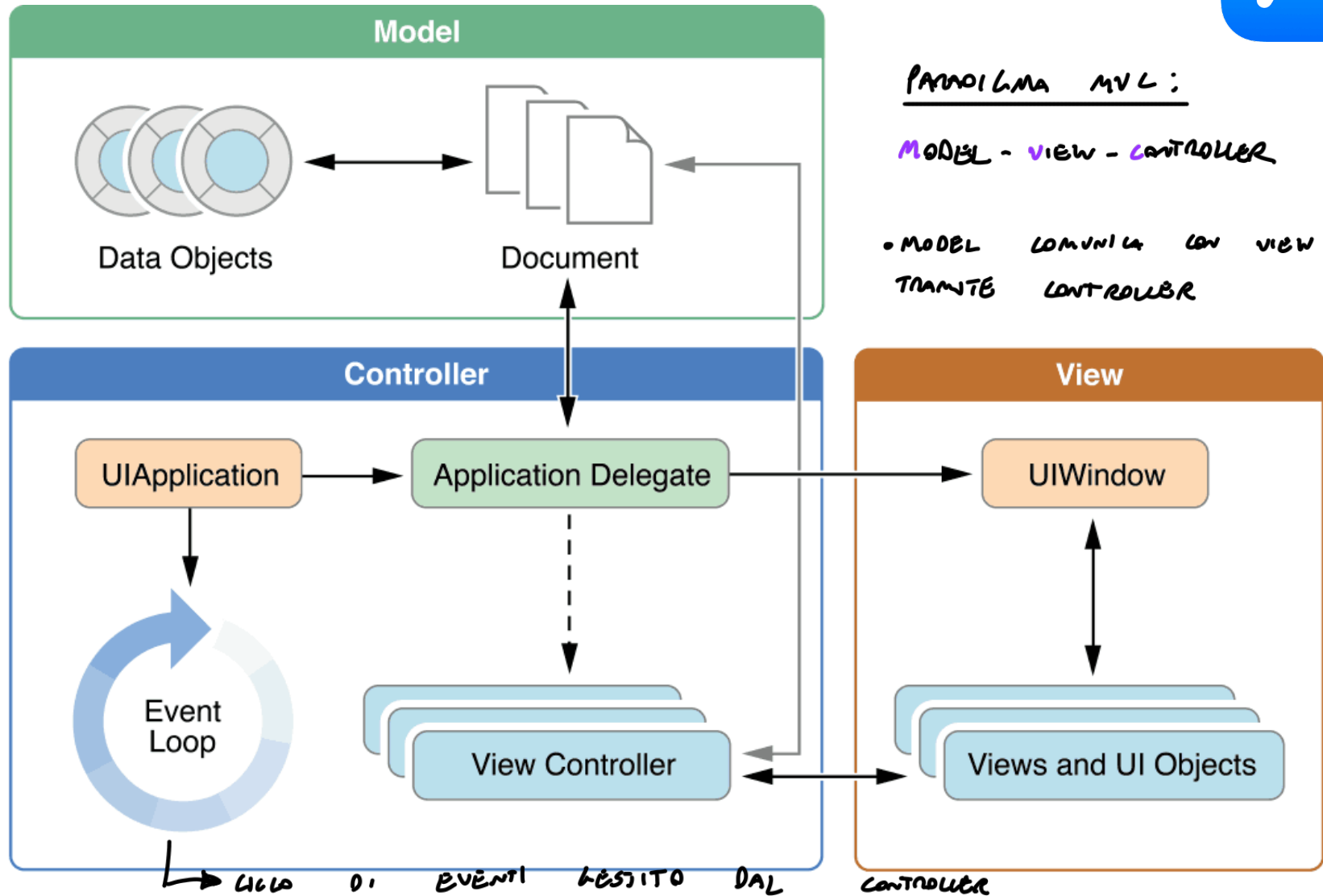
App per dispositivi mobili



DEFINIZIONE APP

- **Pacchetto software** che consente all'utente di eseguire un **task specifico** o una famiglia di task correlati.
 - Gestire la posta, prendere note, partecipare in social network, ecc.
 - Contiene file **eseguibili**, **metadati**, **risorse** ed **asset**.
- Dotata di **GUI** (**G**raphical **U**ser **I**nterface).
 - Tipicamente **semplificata** rispetto ai software desktop.
- Interazione utente tramite **sensoristica integrata**.
 - Touch screen, accelerometro, giroscopio, fotocamera, ecc.
- **Pre-installate** o **scaricabili** tramite opportuni store.
 - Su iOS è (quasi) soltanto possibile tramite App Store.

Architettura di un'app (1/2)



Architettura di un'app (2/2)

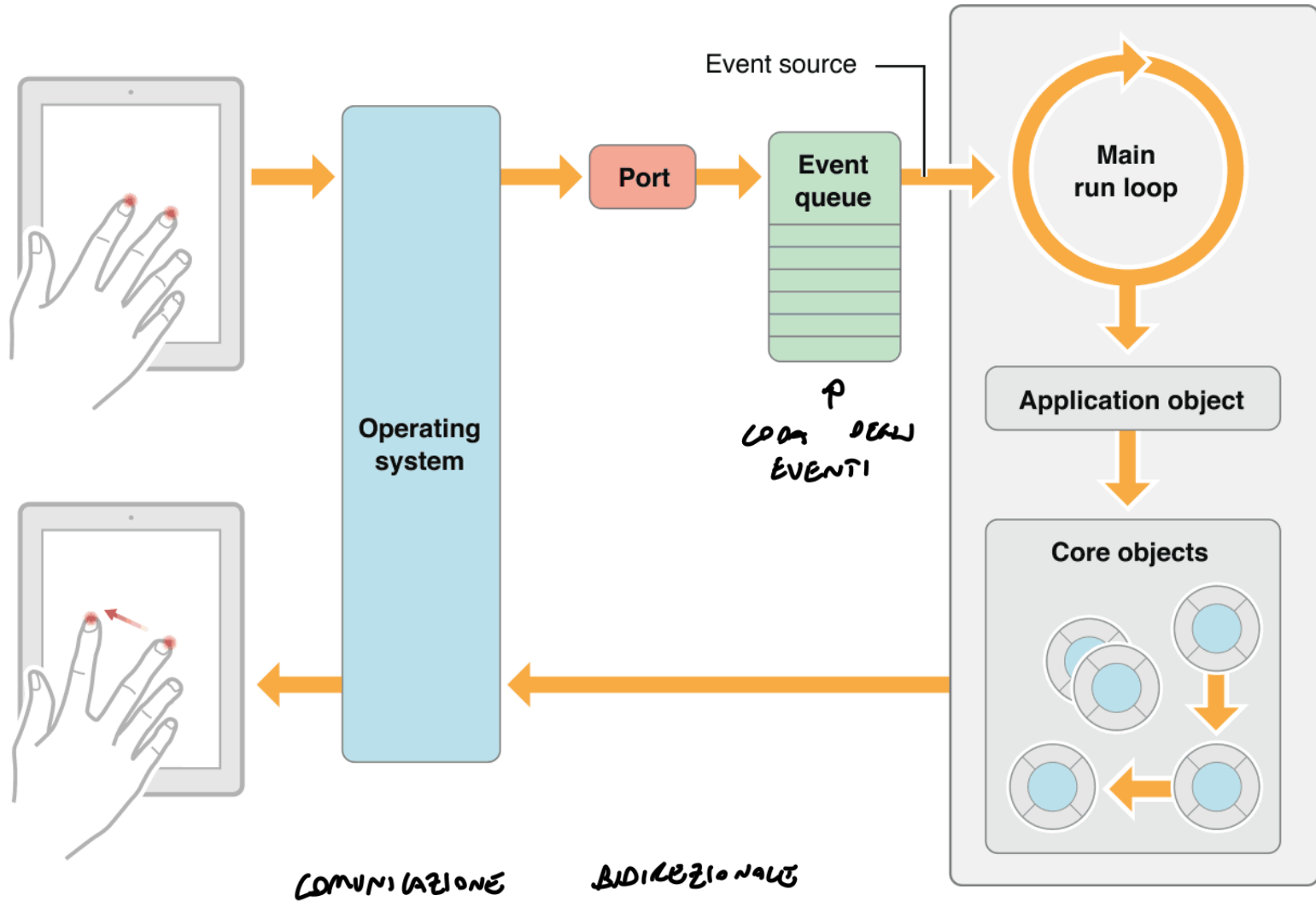


- **View:** componenti relativi all'interfaccia utente.
 - In gergo, una view è un «rettangolo» sullo schermo.
 - Possono essere composte in gerarchie.
 - Built-in o personalizzate dallo sviluppatore.
- **Controller:** intermediario tra model e view.
 - Aggiorna i contenuti di una gerarchia di view, tipicamente a seguito di modifiche nel model.
 - Risponde alle interazioni utente con le view che gestisce coordinandosi con altri oggetti, anche altri controller, per portare a termine task complessi.
- **Model:** dati e logica ad essi relativa.
 - Astrazione degli oggetti di dominio e delle loro relazioni.
 - Es: manipolazione dello stato, accesso alla rete, persistenza, ecc.

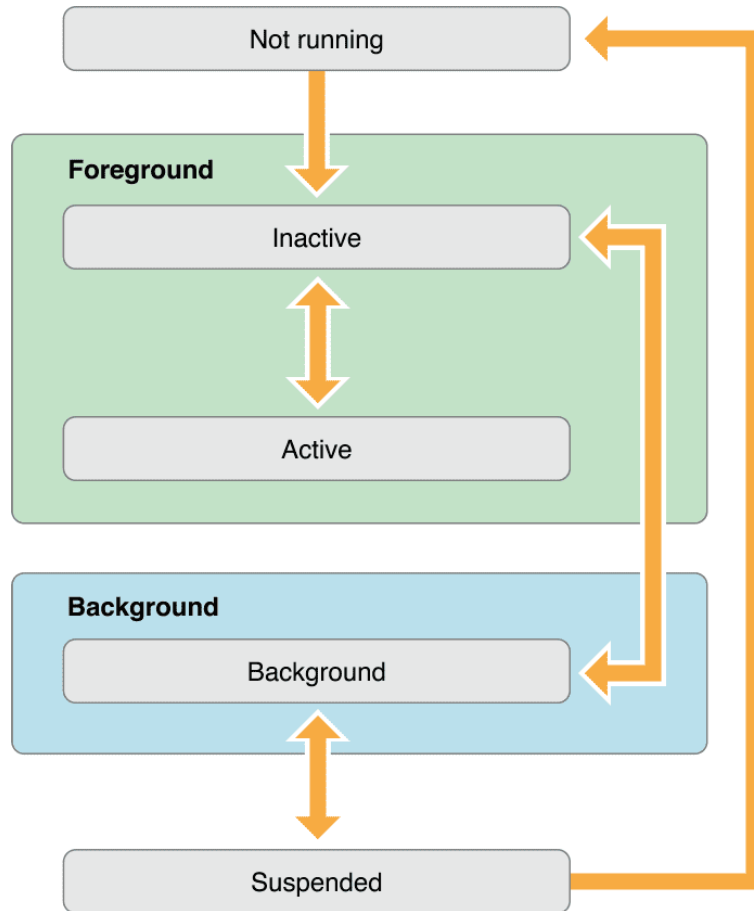
Gestione degli eventi



EVENT LOOP:

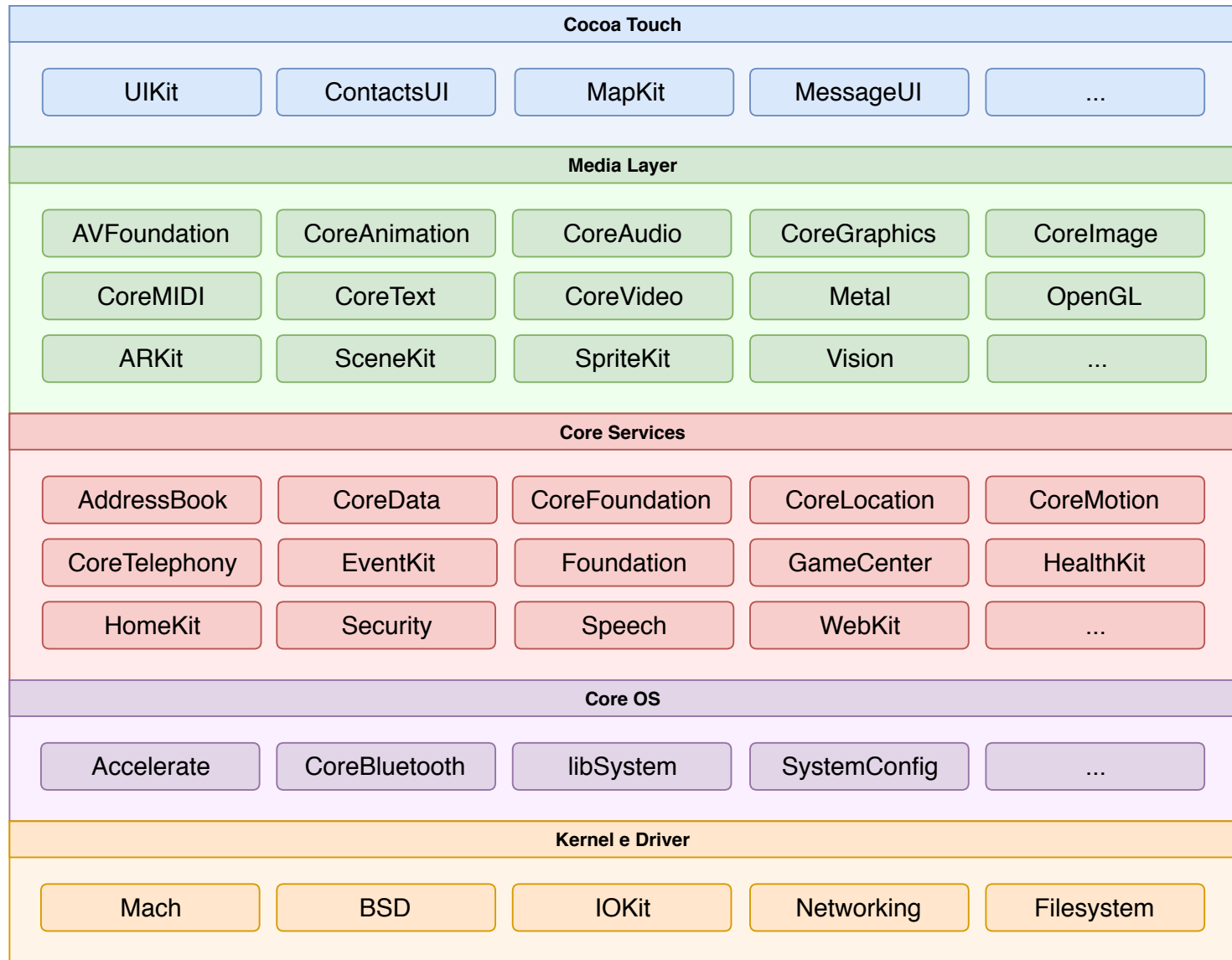


Ciclo di vita di un'app



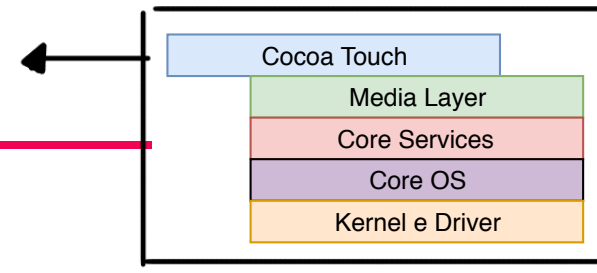
- **Not running:** l'app non è stata lanciata o è stata terminata.
- **Inactive:** l'app è in foreground ma **non riceve eventi** (stato temporaneo di breve durata).
- **Active:** l'app è in foreground e riceve eventi.
- **Background:** l'app è in background e sta eseguendo operazioni.
- **Suspended:** l'app è in background ma non sta eseguendo operazioni.

Stack software



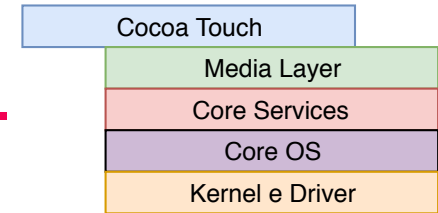
Cocoa Touch

ARCHITETTURA
STACK



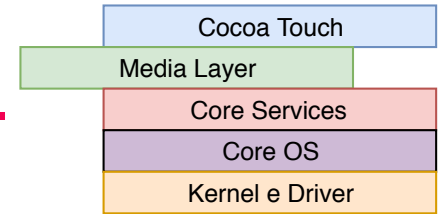
- Il principale framework per lo sviluppo di software su piattaforma iOS.
- Orientato principalmente alle interfacce utente ed all'interazione tramite tocco.
- Scritto in **Objective-C** e basato su **Cocoa** per macOS.
- **UIKit**: sub-framework che fornisce API onnicomprensive per l'implementazione di app native.
- Altri sub-framework forniscono componenti GUI built-in:
 - **ContactsUI**: interfaccia per l'accesso ai contatti.
 - **MapKit**: mappe e geolocalizzazione.
 - **MessageUI**: interfaccia per la composizione di email.
 - ...

Nota: talvolta con «Cocoa Touch» ci si riferisce informalmente ai primi tre livelli.



- Infrastruttura **architetturale** ed **implementativa** per il software iOS:
 - Layout di **finestre** e **view** (pulsanti, etichette, campi di testo, ecc.).
 - **Animazioni** CPU e GPU-accelerated.
 - Riconoscimento di **tocchi** e **gesture**.
 - Controllo del **ciclo di vita** delle applicazioni.
 - Gestione di **risorse** e **asset**.
 - **Condivisione** di file e risorse.
 - Schedulazione e ricezione di **notifiche** locali e push.
 - Rilevazione del **movimento** tramite sensori integrati (accelerometro, giroscopio, prossimità, ecc.).
 - Gestione della **tastiera**.
 - **Inter-app communication**.
 - Feature di **accessibilità**.
 - ...

Media Layer



↳ MANIPOLAZIONE AUDIO, VIDEO...

○ Immagini:

- **CoreImage:** manipolazione di immagini e semplici task di visione artificiale (riconoscimento di volti, testo, codici QR, ecc.)

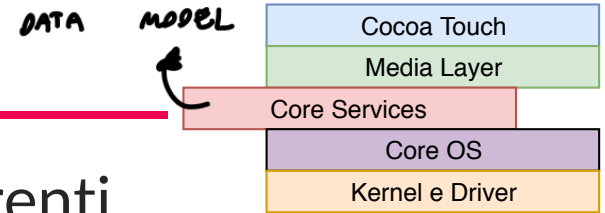
○ Audio/Video:

- **AVFoundation:** supporto di alto livello alla registrazione, modifica e riproduzione di audio e video.
- **CoreAudio/CoreVideo:** elaborazione low-level di stream audio e video, con accesso ai singoli frame video e buffer audio grezzi.

○ Rendering 2D/3D:

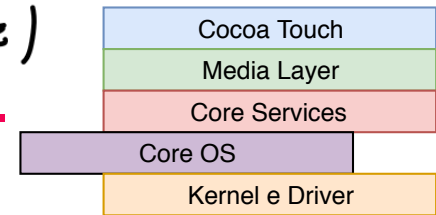
- **CoreGraphics:** API di rendering 2D accelerata tramite CPU.
- **CoreAnimation:** rendering e animazione 2D accelerati tramite *OpenGL*, e più recentemente via *Metal*. Utilizzato da *UIKit* per il rendering delle view.
- **OpenGL/Metal:** API di rendering 3D. *OpenGL* è FOSS e cross-platform, *Metal* è closed e Apple-only, ma offre prestazioni superiori.

Core Services



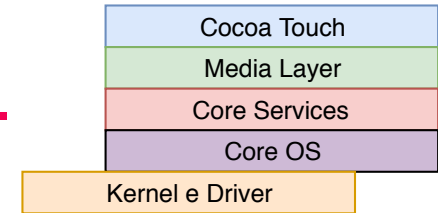
- Servizi essenziali per le app, ma non inerenti all'interfaccia utente:
 - **Foundation:** la «**standard library**» di iOS, fornisce tipi di dato di base (numeri, dati, stringhe, ecc.), collection (liste, set, dizionari, ecc.), accesso al filesystem, algoritmi di filtraggio e ordinamento, ecc.
 - **CoreData:** framework per la gestione e persistenza dei dati, visti come nodi di un grafo. Può essere immaginato come un **ORM** (*Object-Relational Mapping*) con funzionalità avanzate per la gestione del ciclo di vita dei dati. (MODELLO E-R)
 - **CoreLocation:** consente di ottenere la **posizione GPS** e l'**orientazione** del dispositivo relativamente al polo nord magnetico terrestre.
 - **CoreMotion:** permette l'elaborazione di eventi generati da accelerometro, giroscopio, magnetometro, ecc.
 - **WebKit:** motore di rendering di **pagine web**. Consente la visualizzazione di contenuti, la navigazione e la gestione dell'interazione utente con i contenuti visualizzati.

Core OS (livello più basso user land)

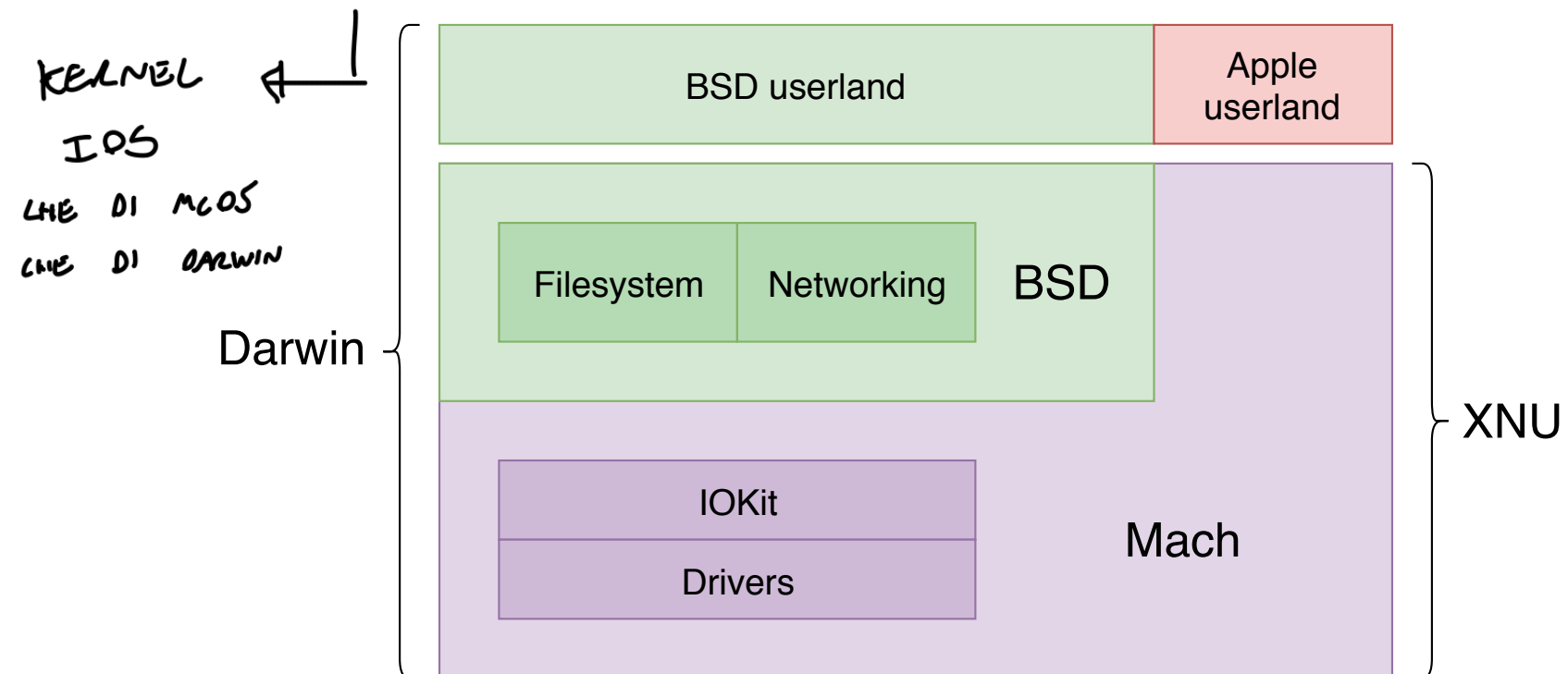


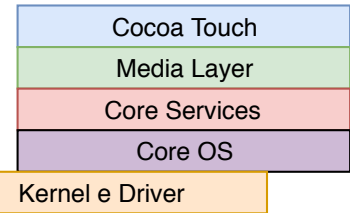
- Il livello più basso dello stack userland, comunicante direttamente con i device driver e con il kernel:
 - **libSystem:** include la **standard library C** ed un subset di API e **syscall** POSIX, Mach e BSD. Fondamenta implementative di tutto il sistema.
 - **Accelerate:** insieme di routine C ottimizzate per il **calcolo numerico**: risoluzione di sistemi di equazioni, calcolo vettoriale e matriciale, DSP (*Digital Signal Processing*), elaborazione di immagini, reti neurali, ecc.
 - **CoreBluetooth:** supporto al discovery ed alla comunicazione a basso livello con dispositivi **Bluetooth Low Energy**.
 - **SystemConfiguration:** consente l'accesso alla **configurazione di rete** del dispositivo, e permette di determinare la disponibilità della connessione internet via WiFi o rete cellulare.

Darwin



- I SO Apple sono basati su **Darwin**, che a sua volta è un sistema operativo completo (non è un kernel).
- Open source, licenza **APSL** (*Apple Public Source License*).
- Kernel **XNU**, userspace **BSD** modificato.





- Acronimo di «**X** is **N**ot **U**nix». «**X**» da **OS X**, la prima versione di Mac OS su cui è stato impiegato XNU.

- **Kernel ibrido**, derivato da **Mach** (*microkernel*) e con importanti aggiunte provenienti da **BSD** (*monolitico*).

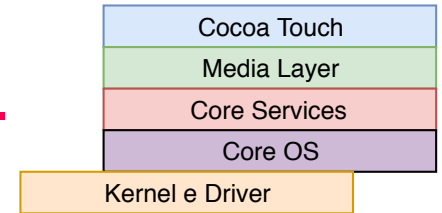
- Perché ibrido?
↗ LINUX MONOLITICO
↗ TUTTE LE FUNZIONALITÀ DEL KERNEL IN KERNEL SPACE

- **Kernel monolitico**: funzionalità core tutte in kernel space e modalità supervisor. Maggiori prestazioni e controllo sull'hardware. Difficili da implementare e mantenere, più facile introdurre bug, affidabilità e sicurezza ridotte.

- **Microkernel**: subset di funzionalità core in kernel mode, il resto ^{USER} delegato a server userspace. Semplici, sicuri e affidabili, ma meno performanti (richiesta di servizio implica IPC e/o più context switch).

- **Kernel ibrido**: modularità e sicurezza derivanti dall'architettura microkernel (Mach), con alcuni servizi performance-critical implementati in kernel space (mutuati da BSD).
↗ MIX TRA MICROKERNEL E MONOLITICO

Mach e BSD (FUN 2102')



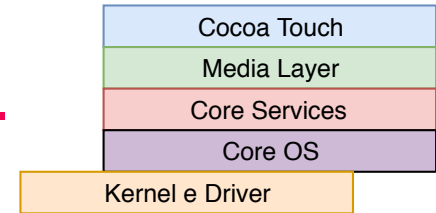
○ Mach:

- **CPU scheduling:** scheduler preemptive basato su code a priorità.
- *Inter Process Communication (IPC)*, basato su message-passing.
- *Remote Procedure Call (RPC)*, basato su Mach IPC.
- Supporto **soft real-time**.
- **Memoria virtuale** paginata.
- **Thread** lato kernel.

○ BSD:

- **Protezione della memoria:** mappatura e protezione delle pagine.
- **VFS** (*Virtual FileSystem*) BSD con filesystem fisici: APFS, HFS/HFS+, NTFS (sola lettura), FAT32/EXFAT e altri.
- **Networking:** stack TCP/IP, socket, firewall, ecc.
- **Modello di sicurezza UNIX:** utenti e gruppi, permessi, policy, ecc.
- API **POSIX** e syscall.
- **Modello processi:** process ID, proprietari, gruppi e segnali.
- **Thread** lato utente (*pthread*) basati su Mach thread.

↑
NON PUBBLICA



- Framework orientato agli oggetti (C++) per lo sviluppo di **device driver**.
- **Privato** su iOS, ma utilizzato a basso livello dal SO. (ABSTRACT)
- Ogni servizio o device è rappresentato da una classe: *avente*
 - **Family**: modella **categorie** di dispositivi (USB, HID, disco, rete, ecc.), permettendo riuso di codice tra device driver di dispositivi appartenenti alla stessa famiglia.
 - **Driver**: oggetto che gestisce una specifica tipologia di **dispositivo**, rappresentandola in maniera astratta. Ciascun driver può appartenere ad una o più family.
 - **Nub**: rappresenta un **canale di comunicazione** con un dispositivo. Tipicamente astrae un bus, buffer o area di memoria mappata per I/O. Ciascun driver espone uno o più nub a seconda dei dispositivi installati che è in grado di controllare.

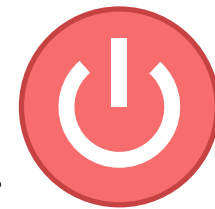
APFS – Apple FileSystem



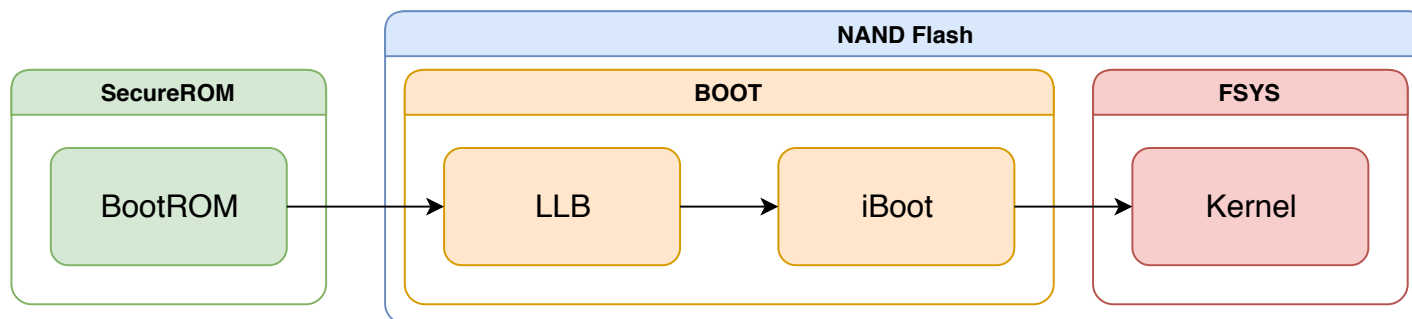
- Proprietario, rilasciato da Apple nel 2016, sostituisce HFS+. Introduce feature moderne quali:
 - **inode** numbers a **64 bit** ($\sim 10^{19}$ file indirizzabili). → INDOE MAPPA I FILE
 - **CoW** (Copy On Write): quando un file viene copiato la sua memoria **non viene duplicata**. Ciò viene fatto solo se si tenta di scrivere su una delle due copie, e solo per i segmenti di file modificati.
 - Rende ridondante il **journaling**: il chunk del file interessato da un'operazione di modifica viene scritto in una nuova area di memoria, e l'inode originale viene aggiornato solo al termine dell'operazione.
 - **Snapshots**: istantanee dei contenuti, che abilitano il ripristino di file o interi volumi ad uno stato noto. Grazie a CoW, gli snapshot **non utilizzano spazio aggizionale** al momento della loro creazione.
 - Supporto a **partizioni di dimensione dinamica** tramite condivisione di *container* di dimensione fissa.
 - **Crittografia**: cifratura a **chiave singola** univoca per file, o a **chiave multipla** per cifrare diverse parti dei file con una chiave diversa.

→ CRITTOGRAFIA DEL FILE SYSTEM

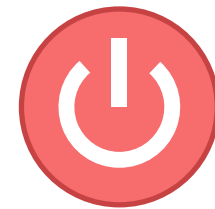
Secure Boot Chain (1)



- L'avvio del dispositivo procede per **stadi**, ciascuno dei quali viene **crittograficamente verificato** dallo stadio precedente, originando il cosiddetto «**boot chain of trust**». In tal modo è possibile assicurarsi che il software di sistema **non sia stato modificato** da terze parti. *↑ STADI TUTTI AUTOGUARATI*
- Stadi dislocati in diverse memorie di massa:
 - **SecureROM**: inaccessibile al kernel, sola lettura.
 - **NAND Flash**: memoria r/w. Diverse partizioni fisiche, tra cui:
 - **BOOT**: inaccessibile al kernel, sovrascrivibile da aggiornamenti software.
 - **FSYS**: contenente il filesystem visibile da kernel e userspace.



Secure Boot Chain (2)



- Il boot chain è così composto:
 - **BootROM**: memoria **read-only**, contiene le prime istruzioni eseguite all'avvio del dispositivo. Viene scritta al momento della produzione ed ha implicitamente la «fiducia» del sistema. Contiene la **root CA key** di Apple, utilizzata per verificare l'integrità degli stadi successivi.
 - **LLB (opzionale)**: *Low Level Bootloader*, presente solo su iPhone 6s e precedenti, esegue diverse subroutine di setup e verifica l'integrità dello stadio successivo.
 - **iBoot**: il bootloader di alto livello, verifica l'integrità del kernel XNU, lo carica in memoria, e ne lancia l'esecuzione.
- Se uno qualsiasi di questi step fallisce, il dispositivo entra in modalità **DFU** (*Device Firmware Update*) o **recovery**, a seconda che il fallimento avvenga a livello di BootROM→LLB→iBoot o iBoot→kernel, rispettivamente.

Feature di sicurezza (1/2)



○ Sicurezza del sistema

- Sicurezza garantita sin dall'avvio tramite **secure boot chain**.
- **Aggiornamenti software** verificati con i server Apple e **firmati digitalmente** tramite chiavi univoche per-device e per installazione.
- Dati biometrici memorizzati in un sottosistema hardware (**SEP, Secure Enclave Processor**) inaccessibile a qualunque processo, kernel incluso. Interazione limitata a message box implementata tramite memoria condivisa.

○ Sicurezza del kernel

- **W^X** (*Write XOR Execute*): pagine di memoria **W** o **X**, mai entrambi.
- **KASLR** (*Kernel Address Space Layout Randomization*): kernel caricato ad indirizzi di memoria casuali ad ogni avvio.
- **KIP** (*Kernel Integrity Protection*): una volta terminato il boot, il memory controller vieta tentativi di scrittura sulla memoria del kernel.
- **PAC** (*Pointer Authentication*): puntatori critici alla memoria sono protetti tramite autenticazione crittografica, verificata in hardware.

Feature di sicurezza (2/2)



○ Sicurezza dei dati

- Se disponibile, iOS fa uso di un **co-processore dedicato** per crittografia AES256.
- **Crittografia** di diverse componenti del sistema tramite chiavi derivate dal passcode dell'utente.
 - Ciascun file è cifrato tramite una (o più) **chiavi uniche**, generate casualmente e cifrate tramite chiavi derivate dal passcode e da ulteriori chiavi «master» presenti nel **SEP**.

○ Sicurezza delle applicazioni

- **Code signing**: le app devono essere firmate crittograficamente da Apple per poter essere lanciate. Firme verificate da **AMFI** (*Apple Mobile File Integrity*), una componente del kernel.
- **Sandboxing**: ciascuna app funziona in una **sandbox**, che limita l'accesso a file di altre app ed a feature per cui non ha autorizzazioni.
- **Entitlement**: l'accesso ad alcune funzionalità (camera, contatti, ecc.) può essere concesso previa richiesta di **consenso all'utente**.

Riferimenti

- **Concetti di Sistemi Operativi**

- A. Silberschatz, P.B. Galvin, G. Gagne,
Operating System Concepts, Wiley

- **Documentazione Apple**

- <https://developer.apple.com/documentation>
- <https://developer.apple.com/library/archive>
- https://www.apple.com/business/docs/iOS_Security_Guide.pdf